

ЛАБОРАТОРНАЯ РАБОТА. Безопасность загрузки ОС

Среда выполнения: виртуальная машина VirtualBox / VMware / QEMU, Debian 13 (Trixie) **Состав работы:** Работа 1 — защита загрузчика GRUB2 (обязательная); Работа 4 — UEFI Secure Boot и подписи модулей ядра (**по желанию**)

Студент: _____ Группа: _____ Дата: _____ Оценка: _____

Виртуальная машина

- **Образ ВМ:** скачать
- **ОС:** Debian 13 (Trixie)
- **Логин:** student
- **Пароль:** qwe123!@#

Как работать с шаблоном

1. Скопируйте документ в Word / LibreOffice / OpenOffice (или откройте выданный преподавателем .docx).
 2. Выполняйте шаги последовательно, набирая команды в терминале ВМ.
 3. Каждый шаг подтверждается **одним** скриншотом (кроме последнего шага работы 4 — там указано два). Блок «МЕСТО ДЛЯ СКРИНШОТА» замените своим снимком экрана и сохраните подпись над ним.
 4. Снимки делайте полноэкранные, чтобы были видны команда и результат.
 5. Перед началом работы создайте снапшот ВМ: эксперименты затрагивают загрузку системы.
-

РАБОТА 1. Защита загрузчика GRUB2

Цель: изучить атаку на процесс загрузки через параметры ядра и настроить аутентификацию в GRUB2.

Краткая теория. GRUB2 перед загрузкой ядра показывает меню, в котором клавишей `e` можно отредактировать пункт: в строке `linux` дописывают `init=/bin/bash`, и система стартует с `root`-оболочкой без аутентификации. Защита строится на механизме `superusers`: при заданном списке суперпользователей редактирование пунктов и консоль GRUB доступны только по логину и паролю (пароль хранится в виде хеша PBKDF2), а штатная загрузка остается открытой через флаг `--unrestricted`.

Шаг 0. Подготовка

Выполните и запишите результат:

```
grub-install --version  
[ -d /sys/firmware/efi ] && echo "Режим загрузки: UEFI" || echo "Режим загрузки: BIOS"
```

Ожидаемый результат: выведена версия GRUB 2.x и режим загрузки.

Шаг 1. Демонстрация атаки (до защиты)

1.1. Перезагрузите ВМ. В меню GRUB нажмите клавишу `e` на пункте Debian. 1.2. В строке, начинающейся с `linux`, допишите в конец `init=/bin/bash` и нажмите `Ctrl+X` (или `F10`). 1.3. В открывшейся оболочке выполните `id` и `mount | grep ' / '`.

Скриншот 1.1 — root-оболочка без аутентификации: виден вывод `id (uid=0)` и монтирование корня.

МЕСТО ДЛЯ СКРИНШОТА 1.1

1.4. Перезагрузите ВМ принудительно (сброс ВМ или `exec /sbin/reboot -f`).

Вывод по шагу 1: _____

Шаг 2. Настройка аутентификации GRUB2

2.1. Сгенерируйте хеш пароля:

```
grub-mkpasswd-pbkdf2
```

Введите пароль дважды (запишите его, он понадобится далее).

Важно. Хеш `grub.pbkdf2.sha512...` индивидуален: он зависит от случайной соли, поэтому у каждого студента получается свой. Скриншот с хешем обязателен — по нему проверяется самостоятельность выполнения работы.

Скриншот 1.2 — вывод `grub-mkpasswd-pbkdf2` с хешем `grub.pbkdf2.sha512...` (виден полностью).

МЕСТО ДЛЯ СКРИНШОТА 1.2

2.2. Запишите настройку суперпользователя в `/etc/grub.d/40_custom` (вместо `grub.pbkdf2...` вставьте свой хеш из п. 2.1):

```
sudo nano /etc/grub.d/40_custom
```

Содержимое файла (после служебных строк `#!/bin/sh` и `exec tail -n +3 $0`):

```
set superusers="admin"
password_pbkdf2 admin grub.pbkdf2.sha512.10000.ВАШ_ХЕШ
```

2.3. Разрешите штатную загрузку без пароля, но запретите редактирование: в файле `/etc/grub.d/10_linux` найдите строки, начинающиеся с `menuentry` ", и добавьте флаг `--unrestricted` после слова `menuentry`:

```
sudo sed -i 's/menuentry "/menuentry --unrestricted "/" /etc/grub.d/10_linux
```

Дополнительно (рекомендуется) отключите `recovery`-пункты в `/etc/default/grub`: `GRUB_DISABLE_RECOVERY=true`.

2.4. Пересоберите конфигурацию загрузчика:

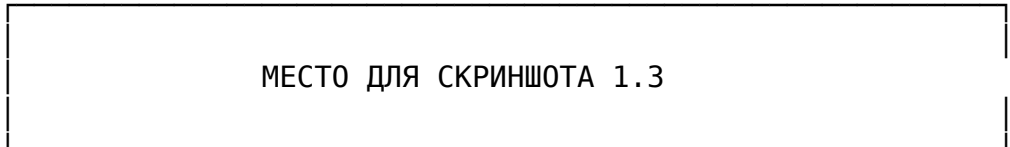
```
sudo update-grub
```

2.5. Перезагрузите ВМ.

Шаг 3. Проверка защиты

3.1. В меню GRUB нажмите е на пункте Debian.

Скриншот 1.3 — запрос логина/пароля GRUB при попытке редактирования (Enter username:).



3.2. Отмените ввод (Esc) и выполните штатную загрузку: выберите пункт Debian и нажмите Enter. Пароль запрашиваться не должен.

3.3. Повторите атаку из шага 1 и убедитесь, что без пароля GRUB она невозможна. Результат опишите в выводах.

Выводы по работе 1

Контрольные вопросы к работе 1

1. Почему добавление `init=/bin/bash` даёт права root без пароля?
2. Чем отличается хранение пароля GRUB в открытом виде от `password_pbkdf2`?
3. Что делает флаг `--unrestricted` и зачем он нужен в этой схеме?
4. Как восстановить доступ к системе, если пароль GRUB утерян?
5. От каких угроз НЕ защищает пароль GRUB (например, загрузка с Live-USB)? Какие средства закрывают эти угрозы?

РАБОТА 4. UEFI Secure Boot и подписи модулей ядра (по желанию)

По желанию. Работа 4 выполняется по желанию. Обязательной для отчёта является работа 1.

Цель: изучить цепочку доверия UEFI Secure Boot, проверить подписи компонентов и зарегистрировать собственный ключ МОК для подписи модуля ядра.

Требования к ВМ: прошивка UEFI и включённый Secure Boot (VMware: Settings → Options → Advanced → Firmware UEFI + флажок Secure Boot; VirtualBox 7: Система →

Материнская плата → EFI и включённый Secure Boot; QEMU: OVMF с secboot). Debian 13 загружается с Secure Boot из коробки: shim, GRUB и ядро подписаны.

Краткая теория. Secure Boot разрешает выполнение только компонентов, подписанных ключами из баз прошивки (PK/KEK/db). Debian использует загрузчик shim, подписанный ключом Microsoft UEFI CA; собственные ключи добавляются через механизм MOK (Machine Owner Key). При включённом Secure Boot ядро находится в режиме lockdown: неподписанные модули не загружаются.

Шаг 0. Подготовка

Установите пакеты:

```
sudo apt update
sudo apt install mokutil sbsigntool build-essential linux-headers-$(uname -r)
```

Шаг 1. Статус Secure Boot и подписи ядра

1.1. Проверьте состояние:

```
mokutil --sb-state
sudo dmesg | grep -iE 'secure boot|lockdown'
```

Ожидаемый результат: SecureBoot enabled; в dmesg — сообщение о режиме lockdown.

Скриншот 4.1 — вывод mokutil --sb-state (Secure Boot включён) и строка о lockdown.

МЕСТО ДЛЯ СКРИНШОТА 4.1

1.2. Проверьте подпись штатного ядра Debian:

```
sbverify --list /boot/vmlinuz-$(uname -r)
```

Шаг 2. Неподписанный модуль отклоняется

2.1. Создайте рабочий каталог и файл модуля hello.c:

```
#include <linux/init.h>
#include <linux/module.h>
#include <linux/kernel.h>

static int __init hello_init(void)
{
    pr_info("Hello, Secure Boot! Modul zagruzhen.\n");
    return 0;
}

static void __exit hello_exit(void)
{
}
```

```

    pr_info("Modul vygruzhen.\n");
}

module_init(hello_init);
module_exit(hello_exit);
MODULE_LICENSE("GPL");
MODULE_DESCRIPTION("Lab module for Secure Boot");
MODULE_AUTHOR("Student");

```

2.2. Создайте Makefile (отступы перед \$(MAKE) — строго символом Tab):

```

obj-m += hello.o
KDIR ?= /lib/modules/$(shell uname -r)/build
all:
    $(MAKE) -C $(KDIR) M=$(PWD) modules
clean:
    $(MAKE) -C $(KDIR) M=$(PWD) clean

```

2.3. Соберите и попробуйте загрузить неподписанный модуль:

```

make
sudo insmod hello.ko

```

Ожидаемый результат: ошибка insmod: ERROR: ... Required key not available.

Скриншот 4.2 — ошибка insmod (Required key not available) после сборки модуля.

МЕСТО ДЛЯ СКРИНШОТА 4.2

Шаг 3. Создание собственного ключа МОК и подпись модуля

3.1. Сгенерируйте ключевую пару (сертификат в формате DER):

```

openssl req -new -x509 -newkey rsa:2048 \
    -keyout MOK.priv -outform DER -out MOK.der \
    -nodes -days 36500 -subj "/CN=Student MOK/"
ls -l MOK.priv MOK.der

```

3.2. Подпишите модуль своим ключом и проверьте подпись:

```

sudo /lib/modules/$(uname -r)/build/scripts/sign-file sha256 MOK.priv MOK.der hello.ko
modinfo hello.ko | grep -E 'sig_id|signer|description'

```

Скриншот 4.3 — вывод modinfo: видны signer: Student MOK и sig_id.

МЕСТО ДЛЯ СКРИНШОТА 4.3

Шаг 4. Регистрация ключа MOK в прошивке

4.1. Импортируйте сертификат (задайте пароль MOK):

```
sudo mokutil --import MOK.der
```

4.2. Перезагрузите ВМ. На старте появится синее меню **Perform MOK management**: выберите Enroll MOK → Continue → Yes → введите пароль MOK → Reboot.

4.3. После загрузки проверьте, что ключ зарегистрирован:

```
mokutil --list-enrolled | grep -B2 -A2 'Student MOK'
```

Скриншот 4.4 — вывод mokutil --list-enrolled с вашим сертификатом (CN=Student MOK).

МЕСТО ДЛЯ СКРИНШОТА 4.4

Шаг 5. Загрузка подписанного модуля

5.1. Загрузите подписанный модуль и проверьте журнал:

```
sudo insmod hello.ko  
sudo dmesg | tail -n 2
```

Ожидаемый результат: модуль загружен, в журнале строка «Hello, Secure Boot!».

Скриншот 4.5 — успешный insmod подписанного модуля и сообщение в dmesg.

МЕСТО ДЛЯ СКРИНШОТА 4.5

5.2. Контрольный опыт: сделайте неподписанную копию (cp hello.ko hello2.ko не подойдёт — подпись останется; пересоберите модуль: make clean && make) и попробуйте sudo insmod hello.ko до подписи — зафиксируйте отказ.

Скриншот 4.6 — отказ загрузки пересобранного неподписанного модуля (контрольный опыт).

МЕСТО ДЛЯ СКРИНШОТА 4.6

5.3. Уберите модуль: sudo rmmod hello.

Выводы по работе 2

Контрольные вопросы к работе 2

1. Постройте цепочку доверия Secure Boot для Debian: от ключей прошивки до ядра.
 2. Что такое shim и зачем он нужен дистрибутивам Linux?
 3. Чем МОК отличается от ключей в базе db прошивки?
 4. Что такое kernel lockdown и какие операции он ограничивает при Secure Boot?
 5. Почему Secure Boot не защищает от кражи диска? Какое средство дополняет его (LUKS, TPM)?
-

ТРЕБОВАНИЯ К ОТЧЁТУ

Отчёт должен содержать:

1. Титульные данные (студент, группа, дата).
2. Цель работы и схему/описание среды (гипервизор, режим прошивки UEFI/BIOS).
3. Ход работы: скриншоты с подписями — **по одному на шаг**:
 - работа 1 (обязательная): 1.1–1.3;
 - работа 4 (по желанию): 4.1–4.6 (в шаге 5 — два снимка).
4. Ответы на контрольные вопросы.
5. Выводы: какие угрозы закрыты выполненными мерами, какие остались и чем закрываются далее (LUKS, TPM, UKI).

ЛИТЕРАТУРА И ССЫЛКИ

- GRUB Manual: <https://www.gnu.org/software/grub/manual/grub/>
- Debian Wiki, SecureBoot: <https://wiki.debian.org/SecureBoot>
- Документация ядра, Module signatures: <https://docs.kernel.org/admin-guide/module-signing.html>
- mokutil: <https://github.com/lcp/mokutil>
- systemd-cryptenroll / ukify: <https://manpages.debian.org/>